

Knuth-Morris-Pratt Algorithm

Christian Rinderknecht

Eszterházy Károly Katolikus Egyetem

September 2026

Basic definitions

An **alphabet** is a non-empty, finite set of symbols, called **letters**, for example 'a', 'b' etc.

A **word** is a finite series of letters from an alphabet, like abracadabra. A letter is a word.

Just as letters can be joined to make up words, so can words: the word $u \cdot v$ is made of the letters of word u followed by the letters of word v , for instance, if $u = \text{back}$ and $v = \text{up}$, then $u \cdot v = \text{backup}$. This operation is called **concatenation**.

Concatenation is associative: $(u \cdot v) \cdot w = u \cdot (v \cdot w)$. The operator may be omitted: $(uv)w = u(vw)$. Concatenation is a non-commutative product, so it has a neutral element ε , called the **empty word**:

$$u \cdot \varepsilon = \varepsilon \cdot u = u.$$

Basic definitions

We denote the **repetition** of a word with an exponent, e.g. $a^3 = aaa$. The **length** of a string x is the length of the associated sequence and is noted $|x|$. So $|\varepsilon| = 0$, $|a| = 1$ and $|a^3| = 3$.

A word x is a **factor** of a word y if there exists two words u and v (possibly empty) such that $y = uxv$. In other words, u occurs in y .

The word x is a **prefix** of y , noted $x \leqslant y$, if $u = \varepsilon$, that is, if $y = xv$. Moreover, it is a **proper prefix**, noted $x < y$, if $v \neq \varepsilon$.

Given $y = uxv$, the word x is a **suffix** of y if $v = \varepsilon$. Furthermore, it is a **proper suffix** if $u \neq \varepsilon$.

Let a range over letters, and x and y over words. The prefix relation is easy to define inductively:

- $\forall y. \varepsilon \leqslant y$;
- $\forall a, x, y. x \leqslant y \Rightarrow a \cdot x \leqslant a \cdot y$.

Basic definitions

A letter in a word can be uniquely characterised by a natural number, called **index**, assuming that the first letter has index 0. Read <https://www.cs.utexas.edu/~EWD/transcriptions/EWD08xx/EWD831.html>

For example, if $x = \text{top}$, then the letter at index 0 is written $x[0] = \text{t}$ and the one at index 2 is $x[2] = \text{p}$.

A factor x of y can be identified by the index of $x[0]$ in y .

The end of the factor can also be given; for example, $x = \text{sit}$ is a factor of $y = \text{curiosity}$ at index 5, written $y[5, 7] = x$. This is sometimes called a **slice** in some programming languages.

Sometimes the upper bound can be left unspecified: $y[i \dots]$, meaning $y[i, |y| - 1]$, if $i < |y|$. That is, $y[i \dots]$ is a **suffix** of y . Likewise: $y[\dots i]$ is a **prefix** of y .

The prefix predicate

An imperative definition:

```
    PREFIX( $x, t$ )  
1   $i \leftarrow 0$   
2  while  $i < \min(|x|, |t|)$   
3      do if  $x[i] = t[i]$   
4          then  $i \leftarrow i + 1$   
5          else  $i \leftarrow |x| + 1$   
6  PREFIX  $\leftarrow i = |x|$ 
```

Does it terminate? What test cases would be meaningful? Is it correct?
Is it optimal?

The prefix predicate

A single static assignment definition:

$\text{PREFIX}(x, t)$

1 $\text{PREFIX} \leftarrow \text{PREFIX}'(0, x, t)$

$\text{PREFIX}'(i, x, t)$

1 **if** $i < \min(|x|, |t|)$

2 **then if** $x[i] = t[i]$

3 **then** $\text{PREFIX}'(i + 1, x, t)$

4 **else** $\text{PREFIX}'(|x| + 1, x, t)$

5 **else** $\text{PREFIX}' \leftarrow i = |x|$

Naïve search

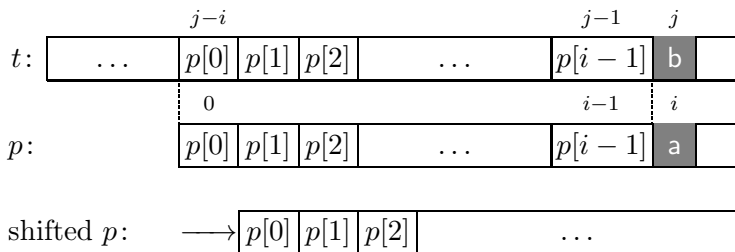
Factor matching is common in text editing, although it is usually better known as **exact string matching** in the academic field of *text algorithmics* (sometimes called *stringology*).

Because of the asymmetric nature of the search, the word p to find will from now on be called the **pattern**, and the word t the **text**.

Assuming p and t are not empty, the naïve search simply consists in checking whether p is a prefix of t : if so, we are done, otherwise, the pattern is shifted by one letter to the right and, again, it is checked whether it is a prefix of that text suffix.

If p cannot be shifted anymore because its end would surpass the end of t , then it is not a factor: the search failed.

Naïve search



Naïvely matching pattern p against text t (failure in grey), where $p[i] \neq t[j]$ (the letters 'a' and 'b' are not relevant in themselves).

Naïve search

An imperative program:

```
    NAIVE( $p, t$ )  
1   $i \leftarrow 0; j \leftarrow 0$   
2  while  $i < |p|$  and  $j < |t|$   
3      do if  $p[i] = t[j]$   
4          then  $i \leftarrow i + 1; j \leftarrow j + 1$   
5          else  $i \leftarrow 0; j \leftarrow j - i + 1$     ▷ We slide  $p$  by 1.  
6   $\text{NAIVE} \leftarrow i = |p|$ 
```

What test cases would be meaningful?

Would it be clearer to use two loops?

How about a version with single static assignments and slices? Without slices?

Naïve search

The input is a pattern p of length m and a text t of length n

What configuration of the input leads to the minimal number of comparisons? What is that number?

In the worst case, p is not in t and the test at line 3 always fails on the last letter of p , leading to make p slide of one position at line 5 the latest as possible.

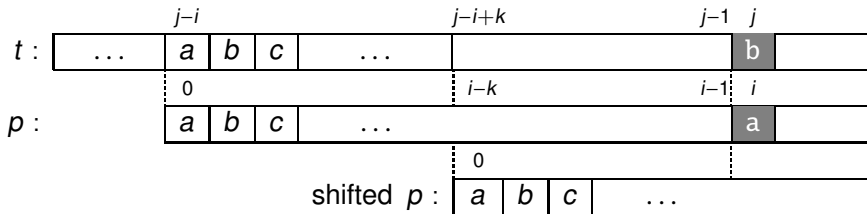
An example of positive match in the worst case is $p = a^{m-1}b$ and $t = a^{n-1}b$, with $m \leq n$.

There are $n - m + 1$ positions in t to compare with the first letter of p , and since there are m letters in p , it makes a total of $(n - m + 1)m = nm - m^2 + m < |t| \times |p|$ positions.

What about the worst case when there is no match?

Morris-Pratt algorithm

In case of mismatch, the naïve algorithm resumes comparing the first letters of p without using the information of the partial success, to wit, we know $p[0, i-1] = t[j-i, j-1]$ and $p[i] \neq t[j]$.

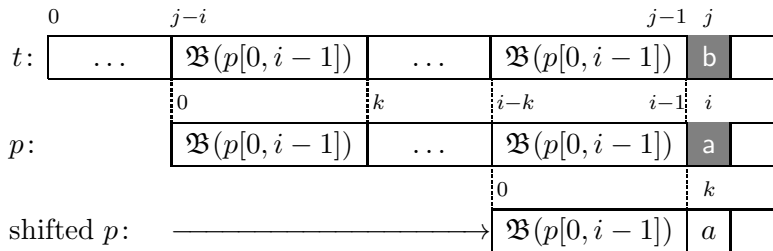


If a new search starts in t at position $j-i+k$, with $1 \leq k \leq i$, it compares a prefix of p to $t[j-i+k \dots j-1]$. Since $t[j-i+k \dots j-1] = p[i-k, i-1]$, it compares a prefix of p to a suffix of $p[\dots i-1]$, that is, to a part of itself!

Morris-Pratt algorithm

The important point here is that these comparisons are *independent* of the text t , since they apply to part of the pattern p itself.

Contrary to naïve factoring, letters in the text are compared in a strictly increasing order, which enables **searches in data streams**.



where $\mathfrak{B}(p)$ is the **maximum border** of p : the longest prefix of p that is also a suffix of $p[\dots i-1]$.

Morris-Pratt algorithm

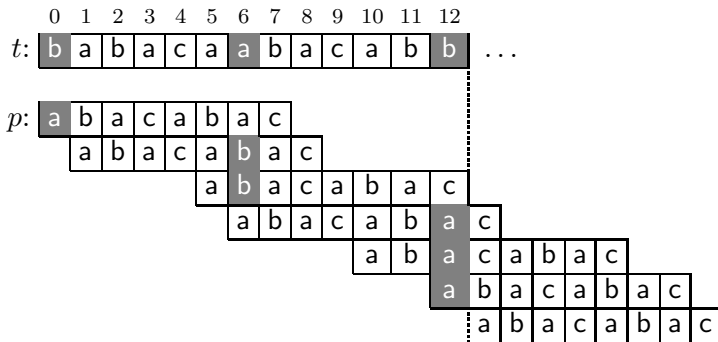
The strategy devised here is to **compute the maximum borders of all prefixes of p before we start the search** in the text, and then use this information to avoid repeating the same comparisons at different positions in the text.

This **preprocessing** on p will indeed allow to recognise the sole configurations where the search may succeed.

In other words, we need to understand borders.

Morris-Pratt algorithm

But before that, let us consider an example where, in the end, p is not found to be a factor of t . As usual, letters on a grey background correspond to mismatches. It is clear that $\mathcal{B}(a) = \varepsilon$, for all letters a .



(The text is not shown in its entirety.)

Morris-Pratt algorithm

The **border** of a non-empty word y is a proper prefix of y which is also a suffix.

To understand the case of the **empty border**, consider $\mathfrak{B}(\text{abac}) = \varepsilon$, simply because we have $\text{abac} = \underline{\varepsilon}\text{abac}\underline{\varepsilon}$.

For example, the word 'abacaba' has three borders: ε , 'a' and 'aba'. The last one is the **maximum border** and we write $\mathfrak{B}(\underline{\text{abacaba}}) = \text{aba}$.

Note that **borders can overlap**; for example,
 $\mathfrak{B}(\underline{\text{aaaa}}) = \mathfrak{B}(\text{aaaa}\underline{}) = \text{aaa}$.

The border above must be a proper prefix, so it cannot equal 'aaaa'.

Morris-Pratt algorithm

As often when working at the level of specifications, we seek an inductive definition for the function at hand, in this instance, the maximum border of a word.

We then have the choice of finding either $\mathfrak{B}(ay)$ or $\mathfrak{B}(ya)$, with $y \neq \varepsilon$, assuming that we know $\mathfrak{B}(y)$.

Since we are interested in knowing the maximum borders of all the *prefixes* of a given pattern, the ya is more suitable ($y < ya$).

The idea is to recursively consider $\mathfrak{B}(y) \cdot a$:

- if it is a prefix of y , then $\mathfrak{B}(ya) = \mathfrak{B}(y) \cdot a$;
- otherwise, we consider the border of $\mathfrak{B}(y) \cdot a$.

Morris-Pratt algorithm

Consider the following examples where y and $\mathfrak{B}(y)$ are given:

$$y = \text{abaabb}, \quad \mathfrak{B}(y) = \varepsilon, \quad \mathfrak{B}(y \cdot b) = \mathfrak{B}(\mathfrak{B}(y) \cdot b) = \mathfrak{B}(b) = \varepsilon;$$

$$y = \text{baaaba}, \quad \mathfrak{B}(y) = \text{ba}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(y) \cdot a = \text{baa};$$

$$y = \text{abbbab}, \quad \mathfrak{B}(y) = \text{ab}, \quad \mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}^2(y) \cdot a = a.$$

For example, $\mathfrak{B}(y \cdot a) = \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^2(y) \cdot a) = \mathfrak{B}^3(y) \cdot a$:

$$y \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}(y) & b & \dots & \mathfrak{B}(y) & a \\ \hline \end{array}$$

$$\mathfrak{B}(y) \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^2(y) & c & \dots & \mathfrak{B}^2(y) & a \\ \hline \end{array}$$

$$\mathfrak{B}^2(y) \cdot a = \begin{array}{|c|c|c|c|} \hline \mathfrak{B}^3(y) & a & \dots & \mathfrak{B}^3(y) & a \\ \hline \end{array}$$

Morris-Pratt algorithm

Formally, for all words $y \neq \varepsilon$ and any letter a ,

$$\mathfrak{B}(a) := \varepsilon; \quad \mathfrak{B}(y \cdot a) := \begin{cases} \mathfrak{B}(y) \cdot a, & \text{if } \mathfrak{B}(y) \cdot a \leqslant y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \text{otherwise.} \end{cases}$$

Unfolding the equations yields:

$$\begin{array}{ll} \mathfrak{B}(ya) = \mathfrak{B}(\mathfrak{B}(y) \cdot a), & \mathfrak{B}(y) \cdot a \not\leqslant y; \\ \mathfrak{B}(\mathfrak{B}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^2(y) \cdot a), & \mathfrak{B}^2(y) \cdot a \not\leqslant \mathfrak{B}(y); \\ \vdots & \vdots \\ \mathfrak{B}(\mathfrak{B}^{p-2}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^{p-1}(y) \cdot a), & \mathfrak{B}^{p-1}(y) \cdot a \not\leqslant \mathfrak{B}^{p-2}(y); \end{array}$$

and $\varepsilon \notin \{y, \mathfrak{B}(y), \dots, \mathfrak{B}^{p-2}(y)\}$, so $\mathfrak{B}(ya) = \mathfrak{B}(\mathfrak{B}^{p-1}(y) \cdot a)$.

If $\mathfrak{B}^{p-1}(y) = \varepsilon$, then $\mathfrak{B}(ya) = \mathfrak{B}(a) = \varepsilon$, else the unfolding resumes until we find the smallest $q > p$ such that $\mathfrak{B}(\mathfrak{B}^{q-1}(y) \cdot a) = \mathfrak{B}(\mathfrak{B}^q(y) \cdot a)$ with $\mathfrak{B}^q(y) \cdot a \leqslant \mathfrak{B}^{q-1}(y)$.

Morris-Pratt algorithm

Because a border is a proper prefix, that is to say, $\mathfrak{B}(y) < y$, we have $\mathfrak{B}^2(y) = \mathfrak{B}(\mathfrak{B}(y)) < \mathfrak{B}(y)$, so $\mathfrak{B}^q(y) \cdot a \leq \mathfrak{B}^{q-1}(y) < \dots < \mathfrak{B}(y) < y$. Therefore $\mathfrak{B}^q(y) \cdot a \leq y$, since $q > 0$, and $\mathfrak{B}(ya) = \mathfrak{B}^q(y) \cdot a$.

Gathering all the cases, an equivalent definition of the maximum border:

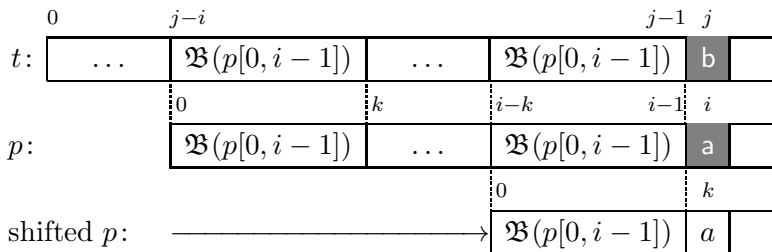
$$\mathfrak{B}(ya) = \begin{cases} \mathfrak{B}^q(y) \cdot a, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ \varepsilon, & \text{otherwise;} \end{cases}$$

with the additional constraint that q **must be as small as possible** (so we define the maximum border).

This form of the definition of $\mathfrak{B}$ is simpler because it does not contain an embedded call.

Morris-Pratt algorithm

We do not actually need to compute the maximum borders of all the prefixes of the pattern. Let us have a look back at



What we need is the offset corresponding to the **length of the maximum border** being considered (k , in the figure).

Morris-Pratt algorithm

Let us define a function N_x on all the prefixes of a word x as follows:

$$N_x(|y|) := |\mathfrak{B}(y)|, \text{ for all } x \text{ and } y \neq \varepsilon \text{ such that } y \leq x. \quad (1)$$

For reasons which will be clear soon, this function is called the **failure function** of x . An equivalent definition is

$$N_x(i+1) = |\mathfrak{B}(x[\dots i])|, \text{ for all } x \text{ and } i \text{ such that } 0 \leq i < |x|.$$

Let's take the equations defining the maximum borders and take the lengths on both sides:

$$|\mathfrak{B}(ya)| = \begin{cases} |\mathfrak{B}^q(y) \cdot a| = 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ |\varepsilon| = 0, & \text{otherwise.} \end{cases}$$

Morris-Pratt algorithm

If $ya \leq x$, then $|\mathfrak{B}(ya)| = \mathcal{N}_x(|ya|) = \mathcal{N}_x(|y| + 1)$. Let $i := |y| > 0$.

$$\mathcal{N}_x(i + 1) = \begin{cases} 1 + |\mathfrak{B}^q(y)|, & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

Let's set aside for a moment $\mathfrak{B}^q(y) \cdot a \leq y$ and return to the definition of $\mathcal{N}$, from which we deduce:

$$\mathcal{N}_x^q(|y|) = |\mathfrak{B}^q(y)|, \text{ with } y \leq x,$$

which we can prove by complete induction on q . Let us call this property $P(q)$. Trivially, we have $P(0)$. Let us suppose $P(n)$ for all $n \leq q$: this is the induction hypothesis. We suppose $y \leq x$ and prove now $P(q + 1)$:

$$\mathcal{N}_x^{q+1}(|y|) = \mathcal{N}_x^q(\mathcal{N}_x(|y|)) = \mathcal{N}_x^q(|\mathfrak{B}(y)|) \doteq |\mathfrak{B}^q(\mathfrak{B}(y))| = |\mathfrak{B}^{q+1}(y)|,$$

where $(\doteq)$ is a valid application of the induction hypothesis because $\mathfrak{B}(y) < y \leq x$. This proves $P(q + 1)$ and the induction principle entails that $P(n)$ holds for all $n \geq 0$.

Morris-Pratt algorithm

So

$$\mathcal{N}_x(i+1) = \begin{cases} 1 + \mathcal{N}_x^q(i), & \text{if } \mathfrak{B}^q(y) \cdot a \leq y; \\ 0, & \text{otherwise.} \end{cases}$$

There is one part of the definition that we did not use: $\mathfrak{B}(a) := \varepsilon$. It implies $\mathcal{N}_x(1) = \mathcal{N}_x(|a|) = |\mathfrak{B}(a)| = |\varepsilon| = 0$ and, since the definition of $\mathcal{N}$ implies $\mathcal{N}_x(1) = 1 + \mathcal{N}_x(0)$, so $\mathcal{N}_x(0) = -1$.

We can now return to the last missing bit:

$$\mathfrak{B}^q(y) \cdot a \leq y \text{ and } ya \leq x \text{ and } |y| = i.$$

Since $\mathfrak{B}^q(y) < y$, those are equivalent to the following:

$$y[|\mathfrak{B}^q(y)|] = a \Leftrightarrow y[\mathcal{N}_x^q(i)] = a \Leftrightarrow x[\mathcal{N}_x^q(i)] = x[|y|] \Leftrightarrow x[\mathcal{N}_x^q(i)] = x[i].$$

Morris-Pratt algorithm

Gathering everything:

$$\mathcal{N}_x(0) = -1 \quad \text{and} \quad \mathcal{N}_x(i+1) = \begin{cases} 1 + \mathcal{N}_x^q(i), & \text{if } x[\mathcal{N}_x^q(i)] = x[i]; \\ 0, & \text{otherwise;} \end{cases}$$

where q is the smallest nonzero natural satisfying the condition.

This can be further simplified into

$$\mathcal{N}_x(0) = -1 \quad \text{and} \quad \mathcal{N}_x(i+1) = 1 + \mathcal{N}_x^q(i), \quad \text{with } i \geq 0,$$

and $q > 0$ is the smallest natural such that $\mathcal{N}_x^q(i) = -1$ or $x[\mathcal{N}_x^q(i)] = x[i]$.

Morris-Pratt algorithm

$\text{NEXT}(x, i)$ $\triangleright \mathcal{N}_x(i)$ with $0 \leq i < |x|$

1 **if** $i = 0$

2 **then** $\text{NEXT} \leftarrow -1$ $\triangleright \mathcal{N}_x(0) = -1$

3 **else** $j \leftarrow i - 1$

4 **repeat** $\triangleright q > 0$ times

5 $j \leftarrow \text{NEXT}(x, j)$

6 **until** $j = -1$ **or** $x[j] = x[i - 1]$ $\triangleright$ Sequentially

7 $\text{NEXT} \leftarrow 1 + j$ $\triangleright \mathcal{N}_x(i) = 1 + \mathcal{N}_x^q(i - 1)$

The table of the maximum borders for the proper prefixes of the word 'abacabac':

$x =$	a	b	a	c	a	b	a	c
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	0	1	0	1	2	3

Morris-Pratt algorithm

Static single-assignment form, assuming $0 \leq i < |x|$:

$\text{NEXT}(x, i)$

```
1  if  $i = 0$   
2    then  $\text{NEXT} \leftarrow -1$   
3    else  $\text{NEXT} \leftarrow 1 + \text{Fix}(x, \text{NEXT}(x, i, i - 1))$ 
```

$\text{Fix}(x, i, j)$

```
1  if  $j = -1$  or  $x[j] = x[i - 1]$   
2    then  $\text{Fix} \leftarrow j$   
3    else  $\text{Fix}(x, i, \text{NEXT}(x, j))$ 
```

Morris-Pratt algorithm

The problem here is inefficiency:

1. successive calls with the same arguments trigger again the same computations,
2. the recursive calls are costly.

It is much preferable to have $\mathcal{N}$ as an array than as a function:

1. the values of $\mathcal{N}$ are computed only once each
2. and are accessed in constant time instead of relying on function calls.

Morris-Pratt algorithm

The idea is to compute all the values of $\mathcal{N}_x(i)$ for *increasing* values of i :

```

NEXT( $x$ )
1   $next[0] \leftarrow -1$ 
2  for  $i \leftarrow 1$  to  $|x| - 1$ 
3      do  $j \leftarrow i - 1$ 
4          repeat
5               $j \leftarrow next[j]$   $\triangleright \mathcal{N}_x[j] < j < i$ 
6              until  $j = -1$  or  $x[j] = x[i - 1]$ 
7               $next[i] \leftarrow 1 + j$   $\triangleright \mathcal{N}_x(i) = 1 + \mathcal{N}_x^q(i - 1)$ 
8   $NEXT \leftarrow next$ 
```

Morris-Pratt algorithm

To find out the best configuration of a pattern p of length m that yields the minimum number of letter comparisons, let us assume that the inner loop in the definition of NEXT is always executed exactly once:

$$\begin{array}{ll} \mathcal{N}_x(m) = 1 + \mathcal{N}_x(m-1) & x[\mathcal{N}_x(m-1)] = x[m-1] \\ \mathcal{N}_x(m-1) = 1 + \mathcal{N}_x(m-2) & x[\mathcal{N}_x(m-2)] = x[m-2] \\ \vdots & \vdots \\ \mathcal{N}_x(2) = 1 + \mathcal{N}_x(1) & x[\mathcal{N}_x(1)] = x[1] \\ \mathcal{N}_x(1) = 1 + \mathcal{N}_x(0) & \\ \mathcal{N}_x(0) = -1 & \end{array}$$

So $\mathcal{N}_x(j) = j - 1$ and $x[j - 1] = x[j]$, for $j > 0$, so the number of letter comparisons for p is $m - 1$. An example of pattern is $p = a^m$.

Morris-Pratt algorithm

To find the worst configuration of a pattern p of length m that yields the maximum number of letter comparisons, let us assume that the inner loop in the definition of `NEXT` is executed as many times as possible, that is to say, the prefix test $x[j - 1] = x[j]$ fails until $j = -1$.

In other words, $N_x(j)$ must decrease as slowly as possible, which is actually the best case: $N_x(j) = j - 1$.

In other words, the worst case is the best case followed by a letter that is not in the best case prefix: $p = a^{m-1}b$, where $a \neq b$. We know that the number of comparisons (equalities) for a^{m-1} is $m - 2$.

We need to add the comparisons (inequalities) of b to all the a : $x[j] \neq x[m - 1]$ for $0 \leq j < m - 1$, that is, $m - 1$.

In total, the number of comparisons in the worst case is $(m - 2) + (m - 1) = 2m - 3$.

Morris-Pratt algorithm

Let us parameterise the search with the failure function:

```

    FIND( $p, t, next$ )
1   $i \leftarrow 0; j \leftarrow 0$ 
2  while  $i < |p|$  and  $j < |t|$ 
3      do if  $i = -1$  or  $p[i] = t[j]$ 
4          then  $i \leftarrow i + 1; j \leftarrow j + 1$ 
5          else  $i \leftarrow next[i]$  ▷  $j$  invariant
6  if  $i = |p|$ 
7      then FIND  $\leftarrow j - |p|$  ▷ Index of first occurrence
8      else FIND  $\leftarrow -1$  ▷ Not found
```

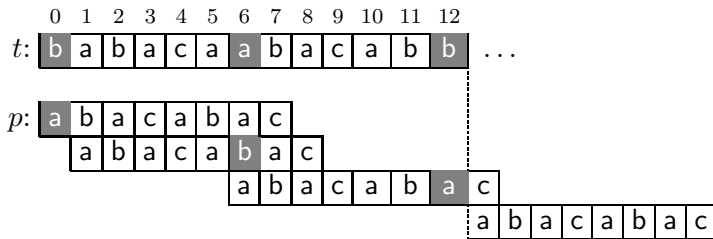
So the Morris-Pratt algorithm is

```

    MP( $p, t$ )
1  MP  $\leftarrow$  FIND( $p, t, NEXT(p)$ )
```

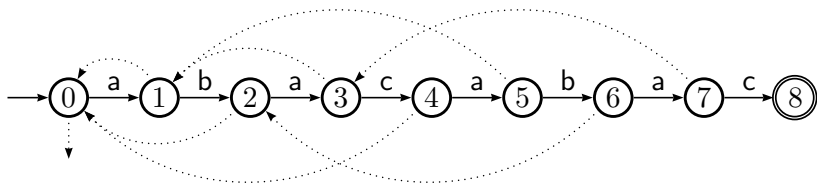
Knuth-Morris-Pratt algorithm

Knuth proposed an improvement: if sliding the pattern entails comparing the same letter from the pattern with the same letter in the text, **we should slide again** because the same mismatch will happen, and again until we find a new letter.



Knuth-Morris-Pratt algorithm

Let us see the need for this improvement in another way, by representing graphically $\mathcal{N}_x$, with $x = \text{abacabac}$, as we know it:



This is the Morris-Pratt *deterministic finite automaton* (DFA) for the word *abacabac*. The table we gave earlier was:

$x =$	a	b	a	c	a	b	a	c
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	0	1	0	1	2	3

Knuth-Morris-Pratt algorithm

We will here only describe informally such automata.

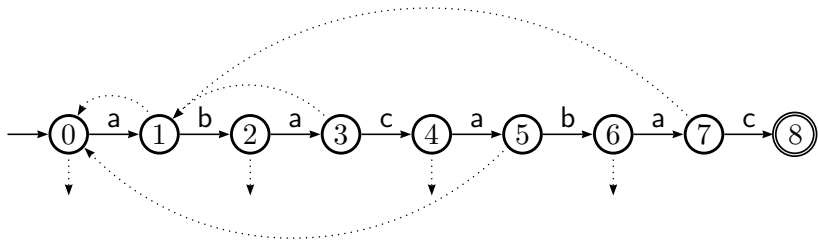
Consider that the circles, called **states**, contain the values of i from the table. The edges, called **transitions**, between two states are of two kinds: either solid and carrying a letter, called **label**, or dotted and going backwards. The sequence of states throughout solid edges make the word $x = \text{abacabac}$.

The rightmost state is distinguished by a double circling because it marks the end of x . There is a back edge between state i and j only if $N_x(i) = j$.

The leftmost state has an incoming, solid edge without a source and a dotted, outgoing edge. The former simply denotes the beginning of the word and the latter corresponds to the special value $N_x(0) = -1$.

Knuth-Morris-Pratt algorithm

In terms of the automaton, the improvement consists in following the back edges



Intuitively, it goes as follows. At a given state with the outgoing label *a*, follow the back edge to another state. If the outgoing label is *a* then repeat, otherwise we are done and we process another state. If the back edge arrives at state 0 and the outgoing label is still *a*, it loses its destination.

Knuth-Morris-Pratt algorithm

In terms of a table, this DFA is equivalent to

$x =$	a b		a c		a b		a c	
i	0	1	2	3	4	5	6	7
$\mathcal{N}_x(i)$	-1	0	-1	1	-1	0	-1	1

$\text{NEXT}'(x)$

```
1  next  $\leftarrow \text{NEXT}(x)$ 
2  for  $i \leftarrow 1$  to  $|x| - 1$ 
3      do  $j \leftarrow \text{next}[i]$ 
4          if  $j \geq 0$  and  $x[j] = x[i]$ 
5              then  $\text{next}[i] \leftarrow \text{next}[j]$ 
6   $\text{NEXT}' \leftarrow \text{next}$ 
```

Knuth-Morris-Pratt algorithm

We can fuse the two loops into one. The idea is to maintain j as the failure state of i before optimisation (that is, original edges $i \rightarrow j$).

```

    NEXT'(x)
1  next[0] ← -1; j ← 0
2  for i ← 1 to |x| - 1
3      do if x[j] = x[i]
4          then next[i] ← next[j]
5          else next[i] ← j;
6              while j ≥ 0 and x[j] ≠ x[i]
7                  do j ← next[j]
8      j ← j + 1
9  NEXT' ← next
```